

# Milestone 2 — Model Development & Training

Project: AI\_PrognosAI — Remaining Useful Life (RUL) Prediction

Date: October 24, 2025

By: K Manjushree (working notebook & code provided)

## 1. Objective

Develop and train a time-series deep-learning model (LSTM / GRU) to learn patterns from multi-sensor turbofan engine data and predict Remaining Useful Life (RUL). Deliverables include the implemented model, trained weights, training/validation loss curves and the complete code for model definition, training and saving.

## 2. Summary of Implementation

The provided implementation uses an LSTM-based architecture with bidirectional and stacked LSTM layers, sequence generation, and feature engineering. Key processes implemented are:

- Data loading from the CMAPSS formatted text files (train/test/RUL for a chosen `FD\_NUMBER`).
- Per-engine RUL computation for training and adjustment of test RULs using the supplied ground truth file.
- Sensor filtering (dropping sensors: sensor\_1, sensor\_5, sensor\_10, sensor\_16, sensor\_18, sensor\_19).
- Feature scaling using `MinMaxScaler`.
- Trend features: per-engine mean/std, diff and rolling mean (window=5).
- Sequence generation for model input using a sliding window (`WINDOW\_SIZE = 30`).
- Model: Bidirectional LSTM layers (128 & 64 units), one unidirectional LSTM (32 units), dense head to a single RUL output.
- Training: 5-fold cross-validation followed by training a final model on the full training set and evaluation on the test set.
- Checkpoints: final model saved as `models\_m2/optimized\_fd{FD\_NUMBER}.h5` and loss curve saved as `graphs\_m2/loss\_curve\_fd{FD\_NUMBER}.png`.

## 3. Key Hyperparameters

| Parameter | Value |

| FD\_NUMBER | 1 |

| Window size (timesteps) | 30 |

| Cross-validation folds | 5 |

| Epochs (max) | 50 |

| Batch size | 64 |

| Optimizer | Adam, lr=0.001 |

| LSTM units | BiLSTM(128) -> BiLSTM(64) -> LSTM(32) |

| Dropout | 0.3 / 0.3 / 0.2 / 0.2 |

| Scaling | MinMaxScaler |

| Trend features | mean, std, diff, rolling\_mean(window=5) |

#### 4. Data & Preprocessing Details

- Files used (based on `BASE\_PATH` in code):
  - `train\_FD001.txt`, `test\_FD001.txt`, `RUL\_FD001.txt` (replace `FD\_NUMBER` as needed).
- Column set: `engine\_id`, `cycle`, 3 operational settings, 21 sensors (original). Several sensors are dropped.
- Missing values: code drops NaNs after reading; verify there were no important missing-value patterns prior to drop.
- Per-engine RUL computation: training RUL is computed as `max\_cycle - cycle` grouped per engine. Test RULs are adjusted by the provided `RUL\_FD00x.txt` ground truth plus the engine's `max\_cycle` offset.
- Feature engineering:
  - Per-engine mean and std per feature (added as features).
  - First differences per engine (`\_diff`) and rolling mean (`\_rolling\_mean`) with window 5.

Recommendation: Save the fitted `scaler` object to disk (pickle) so inference uses identical scaling.

#### 5. Model Architecture (Detailed)

Architecture defined in `build\_lstm\_model`:

1. Input layer (shape = `WINDOW\_SIZE` x `n\_features`)
2. Bidirectional LSTM (128 units) — return\_sequences=True
3. Dropout(0.3)
4. BatchNormalization
5. Bidirectional LSTM (64 units) — return\_sequences=True
6. Dropout(0.3)
7. BatchNormalization
8. LSTM (32 units)
9. Dropout(0.2)
10. Dense(64, activation='relu')
11. Dropout(0.2)
12. Dense(1) — regression output for RUL

Loss: MSE; Metric: MAE. Optimizer: Adam (lr=0.001).

#### 6. Training Strategy

- Validation: 5-fold K-Fold CV on the \*training sequences\* to estimate generalization and tune hyperparameters.
- Callbacks :EarlyStopping (patience 10–15), ReduceLROnPlateau (factor=0.5, patience=5, min\_lr=1e-6).
- Final model: Trained on full training sequences; validated on test sequences (the `X\_test`, `y\_test` generated from test file) and saved.

Note: The current final `fit` uses `validation\_data=(X\_test, y\_test)` which is fine for reporting final test performance, but for hyperparameter search prefer to keep a hold-out validation set separate from final test evaluation.

#### 7. Expected Outputs

- Trained model weights: `models\_m2/optimized\_fd{FD\_NUMBER}.h5`
- Loss curve: `graphs\_m2/loss\_curve\_fd{FD\_NUMBER}.png`

## 8. How to Reproduce

1. Install required packages:

```
```bash  
pip install numpy pandas scikit-learn matplotlib tensorflow keras  
```
```

2. Set `BASE\_PATH` to your dataset folder and run the script.

3. Inspect outputs in `models\_m2` and `graphs\_m2`.

## 9. Observations & Next Steps

- Ensure GroupKFold by engine to prevent data leakage.
- Save scaler and history for reproducible inference.
- Implement hyperparameter tuning and GRU/1D-CNN alternatives.
- Deploy as TensorFlow SavedModel for production use.